

Epreuve Écrite



La clarté de la rédaction et la justification des réponses sont des éléments essentiels de l'appréciation. Les exercices sont indépendants. Le nombre d'exercices est cinq (5) qui sont tous à faire.

Ecrit

On rappelle pour les trois prochains exercices qu'un programme PlusCal peut contenir plusieurs processus et que chaque processus est traduit par une relation next spécifique. Le système global est alors défini par la disjonction des nexts composant le programme. Ensuite, on traduit automatiquement le programme et ses processus en une suite d'actions et de relations next. Une des relations next est utilisée pour le modèle qui produit toutes les exécutions possibles selon la relation next à partir d'une assertion définissant les conditions initiales. Vous avez utilisé l'outil PlusCal et vous avez testé des programme avec plusieurs processus partageant des variables communes. Une instruction particulière est l'instruction `assert A` qui revient à tester à l'exécution si les valeurs courantes des variables satisfont `A`. Comme l'outil engendre toutes les exécutions possibles, si pour une exécution `A` est faux, l'outil stoppera et vous informera de cet état. Dans les exercices 1, 2 et 3 vous devez trouver un condition pour que le `assert` ne soit pas faux pour une exécution possible. Pour cela, vous devez simuler à la main les différents cas d'exécution et proposer une assertion qui convienne. Evidemment si vous utilisez `TRUE`, cela va fonctionner mais cette réponse n'est pas celle attendue et votre assertion doit contenir au moins une occurrence de variables du programme.

Exercice 1 Soit le petit module `pluscalaspd1.tla`.

Listing 1: petit programme `pluscalaspd1`

```

1  ----- MODULE pluscalaspd1 -----
2  EXTENDS Integers, Sequences, TLC, FiniteSets
3  (*
4  --wf
5  --algorithm ex1{
6  variables x = 0;
7
8
9  process (one = 1)
10 {
11   A:
12     x := x - 1;
13 };
14
15 process (two = 2)
16 {
17   C:
18     x := x + 1;
19   D:
20     assert P1;
21 };
22 }
23 end algorithm;
24
25
```

```

26  *)
27  |* BEGIN TRANSLATION
28  VARIABLES x, pc
29
30  vars == << x, pc >>
31
32  ProcSet == {1} |cup {2}
33
34  Init == (* Global variables *)
35         /\ x = 0
36         /\ pc = [self \in ProcSet |-> CASE self = 1 -> "A"
37                [] self = 2 -> "C"]
38
39  A == /\ pc[1] = "A"
40        /\ x' = x + 1
41  ..... /\ pc' = [pc EXCEPT ![1] = "Done"]
42
43  one == A
44
45  C == /\ pc[2] = "C"
46        /\ x' = x + 1
47  ..... /\ pc' = [pc EXCEPT ![2] = "D"]
48
49  D == /\ pc[2] = "D"
50        /\ Assert(...)
51        /\ pc' = [pc EXCEPT ![2] = "Done"]
52  ..... /\ x' = x
53
54  two == C \ / D
55
56  (* Allow infinite stuttering to prevent deadlock on termination. *)
57  Terminating == /\ |A self \in ProcSet: pc[self] = "Done"
58                  /\ UNCHANGED vars
59
60  Next == one \ / two
61         \ / Terminating
62
63  Spec == Init /\ [|Next|_vars
64
65  Termination == <>(|A self \in ProcSet: pc[self] = "Done")
66
67  |* END TRANSLATION
68
69
70
71
72
73
74  =====

```

Donner une expression $P1$ à placer dans la partie `assert` afin que la vérification ne détecte pas d'erreurs dans cette assertion. Par exemple, on pourrait proposer $x = 1 \vee x = 2$ mais il vous appartient de simuler le programme pluscal pour vérifier que jamais l'assertion que vous proposerez ne soit fausse. La solution `TRUE` fonctionne mais n'est pas autorisée et l'expression demandée doit contenir une occurrence de x au moins.

Exercice 2 Soit le petit module `pluscalaspd2.tla`.

Listing 2: petit programme `pluscalaspd2`

```

1  ----- MODULE pluscalaspd2 -----
2  EXTENDS Integers, Sequences, TLC, FiniteSets
3  (*
4  --wf
5  --algorithm ex1{
6  variables x = 0;
7
8
9  process (one = 1)
10 {
11   A:
12     x := x + 1;
13   B:
14     x := x + 1;
15 };
16

```

```

17 process (two = 2)
18 {
19   C:
20     x := x - 1;
21   D:
22     assert !{1};
23 };
24
25 }
26 end algorithm;
27
28 *)
29 \* BEGIN TRANSLATION
30 VARIABLES x, pc
31
32 vars == << x, pc >>
33
34 ProcSet == {1} \cup {2}
35
36 Init == (* Global variables *)
37   /\ x = 0
38   /\ pc = [self \in ProcSet -> CASE self = 1 -> "A"
39           [] self = 2 -> "C"]
40
41 A == /\ pc[1] = "A"
42      /\ x' = x - 1
43      /\ pc' = [pc EXCEPT ![1] = "B"]
44
45 B == /\ pc[1] = "B"
46      /\ x' = x + 1
47      /\ pc' = [pc EXCEPT ![1] = "Done"]
48
49 one == A /\ B
50
51 C == /\ pc[2] = "C"
52      /\ x' = x - 1
53      /\ pc' = [pc EXCEPT ![2] = "D"]
54
55 D == /\ pc[2] = "D"
56      /\ Assert P2
57      /\ pc' = [pc EXCEPT ![2] = "Done"]
58      /\ x' = x
59
60 two == C /\ D
61
62 (* Allow infinite stuttering to prevent deadlock on termination. *)
63 Terminating == /\ |A self \in ProcSet: pc[self] = "Done"
64                  /\ UNCHANGED vars
65
66 Next == one /\ two
67         /\ Terminating
68
69 Spec == Init /\ [|Next]_vars
70
71 Termination == <>(|A self \in ProcSet: pc[self] = "Done")
72
73 \* END TRANSLATION
74
75
76
77
78
79
80

```

Donner une expression P2 à placer dans la partie `assert` afin que la vérification ne détecte pas d'erreurs dans cette assertion. Par exemple, on pourrait proposer $x = 1 \vee x = 2$ mais il vous appartient de simuler le programme pluscal pour vérifier que jamais l'assertion que vous proposerez ne soit fausse. La solution TRUE fonctionne mais n'est pas autorisée et l'expression demandée doit contenir une occurrence de x au moins.

Exercice 3 Soit le petit module `pluscalaspd3.tla`.

Listing 3: petit programme `pluscalaspd3`

```

2 | EXTENDS Integers, Sequences, TLC, FiniteSets
3 | (*
4 | --wf
5 | --algorithm ex3{
6 |   variables x = 0, y = 2;
7 |
8 |   process (one = 1)
9 |   {
10 |     A:
11 |       x := x + 1;
12 |     B:
13 |       y := y - 1;
14 |     C:
15 |       assert Q1;
16 |   };
17 |
18 |   process (two = 2)
19 |   {
20 |     D:
21 |       x := x - 1;
22 |     E:
23 |       y := y + 2;
24 |     F:
25 |       x := x + 2;
26 |     G:
27 |       assert Q2;
28 |   };
29 |
30 | }
31 | end algorithm;
32 |
33 | *)
34 | \* BEGIN TRANSLATION
35 | VARIABLES x, y, pc
36 |
37 | vars == << x, y, pc >>
38 |
39 | ProcSet == {1} \cup {2}
40 |
41 | Init == (* Global variables *)
42 |   /\ x = 0
43 |   /\ y = 2
44 |   /\ pc = [self \in ProcSet -> CASE self = 1 -> "A"
45 |           [] self = 2 -> "D"]
46 |
47 | A == /\ pc[1] = "A"
48 |      /\ x' = x + 1
49 |      /\ pc' = [pc EXCEPT ![1] = "B"]
50 |      /\ y' = y
51 |
52 | B == /\ pc[1] = "B"
53 |      /\ y' = y - 1
54 |      /\ pc' = [pc EXCEPT ![1] = "C"]
55 |      /\ x' = x
56 |
57 | C == /\ pc[1] = "C"
58 |      /\ Assert(...)
59 |      /\ pc' = [pc EXCEPT ![1] = "Done"]
60 |      /\ UNCHANGED << x, y >>
61 |
62 | one == A /\ B /\ C
63 |
64 | D == /\ pc[2] = "D"
65 |      /\ x' = x - 1
66 |      /\ pc' = [pc EXCEPT ![2] = "E"]
67 |      /\ y' = y
68 |
69 | E == /\ pc[2] = "E"
70 |      /\ y' = y + 2
71 |      /\ pc' = [pc EXCEPT ![2] = "F"]
72 |      /\ x' = x
73 |
74 | F == /\ pc[2] = "F"
75 |      /\ x' = x + 2
76 |      /\ pc' = [pc EXCEPT ![2] = "G"]
77 |      /\ y' = y
78 |
79 | G == /\ pc[2] = "G"
80 |

```

```

81  /| Assert((...))
82  /| pc' == pc EXCEPT !|2| == "Done" |
83  ===== /| UNCHANGED << x, y >>
84
85  two == D_ | / E_ | / F_ | / G
86
87  (* Allow infinite stuttering to prevent deadlock on termination. *)
88  Terminating == /| A_self | in ProcSet: pc/self == "Done"
89  ===== /| UNCHANGED vars
90
91  Next == one | / two
92  ===== /| Terminating
93
94  Spec == Init_ | / || Next | _vars
95
96  Termination == <> (A_self | in ProcSet: pc/self == "Done")
97
98  |*_END_TRANSLATION
99
100
101
102
103
104
105

```

Donner les deux expressions Q1 et Q2 à placer dans les parties assert afin que la vérification ne détecte pas d'erreurs dans cette assertion. Par exemple, on pourrait proposer $(x = 1 = 2) \wedge (y = 0 \vee y = 5)$ mais il vous appartient de simuler le programme pluscal pour vérifier que jamais l'assertion que vous proposerez ne soit fausse. La solution TRUE fonctionne mais n'est pas autorisée et les expressions demandées doivent contenir une occurrence de x au moins et une occurrence de y .

Exercice 4 (Exclusion mutuelle)

Vous avez eu les présentations des protocoles de demande de la section critique notamment Lamport et Ricart-Agrawala. Expliquez simplement les différences entre les deux protocoles notamment le nombre de messages nécessaires pour obtenir une section critique.

Exercice 5 (protocoles de communication)

L'Internet des Objets repose sur des protocoles de communication qui doivent obéir à des contraintes de fiabilité dans un milieu bruyé, voire hostile. L'un des protocole utilisé dans les solutions pratiques est appelé LoRAWAN et utilise des protocoles plus élémentaires de communication entre les différentes entités. Le schéma ci-après décrit les flux de communication:

- entre sensors et Base station
- entre Base station et Gateways
- entre Gateways et Network server
- entre Network et server application servers

On demande de proposer des protocoles de communication entre Base station et Gateways et entre Gateways et Network server. Vous devez proposer une ou plusieurs solutions en expliquant que vos solutions permettent de ne pas perdre de messages.

